Digital VLSI Design

4-bit Array Multiplier from RTL to GDSII with Open-Source Tools

Contents

1 Overview **1**

2 Background: How a 4-bit Array Multiplier Works **1**

3 Open-Source Tools and Links **3**

4 Environment Setup **3**

5 File Architecture **4**

6 Part A — RTL Design and Verification **6**

7 Part B — Logic Synthesis with Yosys **7**

8 Part C — Static Timing Analysis with OpenSTA **9**

9 Part D — Physical Design: RTL to GDSII **11**

10 Questions for the Report **13**

11 Deliverables **13**

12 Grading **13**

13 Academic Integrity **13**

14 Troubleshooting **14**

15 References **14**

1. Overview

You will design a 4-bit unsigned array multiplier in Verilog, verify it, and take it through a complete open-source RTL-to-GDSII flow on the SkyWater 130 nm process: synthesis, static timing analysis, and place and route to a final layout. You will then explain your results using design concepts.

Item	Detail
Weightage	15% of total coursework
Type	Team assignment (2 students per team)
Duration	2 weeks
Design	4-bit × 4-bit unsigned array multiplier (8-bit product)
Technology	SkyWater SKY130 (130 nm), standard cell library `sky130_fd_sc_hd`
Platform	Linux, or Windows/macOS with Docker

Your team writes every file itself. Section 5 lists exactly which files you must create and what each one must contain.

2. Background: How a 4-bit Array Multiplier Works

Read this section before writing any Verilog.

Partial products

Multiplying two 4-bit numbers A and B is shift-and-add. For every bit b_i of B there is one partial product: A AND b_i (every bit of A ANDed with b_i), shifted left by i places. Adding the four partial products gives the 8-bit product.

Example: $A = 1011$ (11) and $B = 1101$ (13).

```
          1 0 1 1     A
        x 1 1 0 1     B
---------------
          1 0 1 1     pp0 = A AND b0 (b0 = 1)
        0 0 0 0       pp1 = A AND b1 (b1 = 0), shifted 1
      1 0 1 1         pp2 = A AND b2 (b2 = 1), shifted 2
    1 0 1 1           pp3 = A AND b3 (b3 = 1), shifted 3
---------------
1 0 0 0 1 1 1 1     P = 143
```

The array structure

The hardware adds the partial products one row at a time.

1. **Row 0** is only AND gates. Its lowest bit is already the final product bit $p[0]$; the other three bits are passed down to row 1.
2. **Rows 1, 2 and 3** are each a chain of 4 full adders (a 4-bit ripple-carry adder). Row i adds partial product pp_i to the 4 bits passed down from row $i - 1$.
3. The **lowest sum bit** of each row is a finished product bit ($p[1]$, $p[2]$, $p[3]$). The other three sum bits and the final carry are passed to the next row.
4. The **last row** (row 3) also produces the top four product bits $p[7{:}4]$.

```
  a[3:0] & b[0]  --> row 0 (4 AND gates)                    --> p[0]
```

```
a[3:0] & b[1]  --> row 1 (4 AND gates + 4 full adders)    --> p[1]
a[3:0] & b[2]  --> row 2 (4 AND gates + 4 full adders)    --> p[2]
a[3:0] & b[3]  --> row 3 (4 AND gates + 4 full adders)    --> p[3] and p[7:4]
```

In total the multiplier uses **16 AND gates and 12 full adders**. In general, an N-bit array multiplier uses N^2 AND gates and $N(N-1)$ full adders, and its longest path is roughly $2N$ full-adder delays, so the delay grows about linearly with N.

Exact connections

Use these equations to write `mult_array.v`. Here $pp_i[j] = a[j]$ AND $b[i]$.

Row	**Inputs to full adder j ($j = 0 \ldots 3$)**	**Outputs**
0	no adders	$p[0] = pp_0[0]$; passes down $r_0 = \{0,\ pp_0[3],\ pp_0[2],\ pp_0[1]\}$
1	$pp_1[j]$, $r_0[j]$, carry $c_1[j]$ ($c_1[0] = 0$)	sum $s_1[j]$, carry $c_1[j{+}1]$; $p[1] = s_1[0]$; passes down $r_1 = \{c_1[4],\ s_1[3],\ s_1[2],\ s_1[1]\}$
2	$pp_2[j]$, $r_1[j]$, carry $c_2[j]$ ($c_2[0] = 0$)	sum $s_2[j]$, carry $c_2[j{+}1]$; $p[2] = s_2[0]$; passes down $r_2 = \{c_2[4],\ s_2[3],\ s_2[2],\ s_2[1]\}$
3	$pp_3[j]$, $r_2[j]$, carry $c_3[j]$ ($c_3[0] = 0$)	sum $s_3[j]$, carry $c_3[j{+}1]$; $p[3] = s_3[0]$; $p[7{:}4] = \{c_3[4],\ s_3[3],\ s_3[2],\ s_3[1]\}$

Registers and latency

The multiplier registers its inputs and its output, so timing is measured from one flip-flop to the next. An input applied before clock edge 1 is captured into the input registers at edge 1; the product is captured into the output register at edge 2. The **latency is therefore 2 clock cycles**. **Throughput** is one result per clock cycle, so it equals the clock frequency.

3. Open-Source Tools and Links

All tools are free and open source, and all of them come pre-installed in the IIC-OSIC-TOOLS Docker image (Section 4).

Flow stage	Tool	Official link	Used in
All-in-one environment	IIC-OSIC-TOOLS	github.com/iic-jku/IIC-OSIC-TOOLS	Setup
RTL and gate-level simulation	Icarus Verilog	steveicarus.github.io/iverilog	A, B
Waveform viewing	GTKWave	gtkwave.github.io/gtkwave	A
Logic synthesis	Yosys	yosyshq.net/yosys	B
Static timing analysis	OpenSTA	github.com/parallaxsw/OpenSTA	C
RTL-to-GDSII flow	LibreLane (successor to OpenLane 2)	github.com/librelane/librelane	D
Place and route engine	OpenROAD	theopenroadproject.org	D
Layout viewing and DRC	KLayout	klayout.de	D
Layout DRC and extraction	Magic	opencircuitdesign.com/magic	D
LVS	Netgen	opencircuitdesign.com/netgen	D
Process design kit	SkyWater SKY130	github.com/google/skywater-pdk, documentation	All
Docker runtime	Docker Desktop	docker.com/products/docker-desktop	Setup

Note on OpenLane. OpenLane 1 is in maintenance mode. OpenLane 2 continues under the FOSSi Foundation as **LibreLane**, which accepts the same kind of configuration file, so most OpenLane tutorials still apply. Announcement

4. Environment Setup

Use the IIC-OSIC-TOOLS Docker image, which bundles every tool and the SKY130 PDK. The image is about 4 GB to download; keep at least 20 GB free disk space and give Docker at least 8 GB RAM.

4.1 Install and launch

1. Install Docker Desktop. Windows users: enable virtualization in BIOS and install WSL 2 first.
2. Follow the "Quick launch" instructions in the IIC-OSIC-TOOLS README to start the container in browser (VNC) mode, then open `http://localhost` in a browser.
3. Your files live in `~/eda/designs` on your computer, which appears as `/foss/designs` inside the container. Always work inside this folder, or your work is lost when the container stops.

4.2 Check the tools

Run these in a terminal inside the container. Each should print a version number.

```
iverilog -V | head -1
yosys -V
sta -version
echo $PDK_ROOT $PDK        # should show sky130A; if not, run:  sak-pdk sky130A
```

The two library files you will need are:

File	Path inside the container
Liberty timing library (typical corner)	`$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/lib/sky130_fd_sc_hd__tt_025C_1v80.lib`
Standard cell Verilog models	`$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog/primitives.v` and `sky130_fd_sc_hd.v` in the same folder

5. File Architecture

Organise it with exactly this structure. Use exactly these file names, module names and port names, because the checking is done against them.

```
<team-repository>/
|-- README.md
|-- rtl/
|   |-- full_adder.v
|   `-- mult_array.v
|-- tb/
|   `-- tb_mult.v
|-- synth/
|   |-- synth.tcl
|   |-- mult_array_netlist.v        (generated)
|   `-- mult_array_stat.rpt         (generated)
|-- sta/
|   |-- sta.tcl
|   `-- mult_array_timing.rpt       (generated)
|-- pnr/
|   |-- config.json
|   |-- metrics.json                (copied from the final run)
|   `-- mult_array.gds              (copied from the final run)
`-- report/
    `-- report.pdf
```

What each file must contain

File	Required contents	Part
`README.md`	Names and roll numbers of both team members, and the exact command you ran for every step, in order	All
`rtl/full_adder.v`	Module `full_adder` with ports `a`, `b`, `cin` (inputs) and `sum`, `cout` (outputs), written with `assign` statements	A
`rtl/mult_array.v`	Module `mult_array` with the ports listed below. It must use 12 instances of `full_adder` for the three adder rows. The `*` operator is **not allowed**	A
`tb/tb_mult.v`	Module `tb_mult`: an exhaustive, self-checking testbench as specified in Part A	A
`synth/synth.tcl`	Yosys script with the steps listed in Part B	B
`sta/sta.tcl`	OpenSTA script with the steps listed in Part C	C
`pnr/config.json`	LibreLane configuration with the settings listed in Part D	D
`pnr/metrics.json`, `pnr/mult_array.gds`	Copied from the `final/` folder of your successful LibreLane run	D
`report/report.pdf`	Your completed results tables, screenshots and answers to the questions in Section 10	All

Ports of `mult_array`

Port	Direction	Width	Description
`clk`	input	1	Clock, rising edge
`rst_n`	input	1	Asynchronous reset, active low: clears all registers to 0
`a`	input	4	Multiplicand
`b`	input	4	Multiplier
`p`	output	8	Product $a \times b$, registered

Inside `mult_array`, register `a` and `b` into 4-bit registers on every rising clock edge, build the array from the registered values using the table in Section 2, and register the 8-bit result into `p`. This gives exactly **16 flip-flops** and a latency of 2 clock cycles.

6. Part A — RTL Design and Verification

Testbench requirements

Your `tb/tb_mult.v` must:

- generate a clock with a 10 ns period and hold `rst_n` low for the first 2 cycles;
- apply **all 256 input combinations** ($a = 0 \ldots 15$, $b = 0 \ldots 15$);
- for each combination, wait for the 2-cycle latency and compare `p` with the expected value `a * b` (the `*` operator is allowed in the testbench only);
- print a line for every mismatch, and at the end print exactly one summary line in this format: `PASS: 256 vectors checked, 0 errors` (or `FAIL: ...` with the error count);
- write a waveform file `mult.vcd`.

Tasks

1. Write `rtl/full_adder.v`.
2. Write `rtl/mult_array.v` following Sections 2 and 5. You may instantiate the 12 full adders one by one or with `generate` loops.
3. Write `tb/tb_mult.v` meeting the testbench requirements above.
4. Simulate and fix your design until the testbench prints PASS:

```
iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
vvp sim
gtkwave mult.vcd
```

5. Choose this as your example: A = the last digit of either team member's roll number $+5$ (say whose roll number you used), and $B = 7$ (if $A = 7$, use $B = 9$). Work out $A \times B$ by hand in the same layout as the example in Section 2, and find the same multiplication in your waveform.
6. Check that your testbench really catches errors: temporarily change `sum` in `full_adder.v` to `a ^ b`, run the testbench, and confirm it prints FAIL. Then undo the change.

Part A deliverables

- `rtl/full_adder.v`, `rtl/mult_array.v`, `tb/tb_mult.v`.
- The PASS summary line, and the FAIL line from task 6.
- A waveform screenshot showing `clk`, `rst_n`, `a`, `b` and `p` for your example from task 5, with the 2-cycle latency marked.
- Your hand-worked example from task 5.
- A block diagram of your multiplier showing the 16 AND gates, 12 full adders and the registers (hand-drawn and scanned is fine).

7. Part B — Logic Synthesis with Yosys

Yosys converts your RTL into a netlist of SKY130 standard cells. Manual: Yosys documentation.

Steps your `synth.tcl` must perform

Each step is one Yosys command inside a Tcl script (in a Tcl script, write each command as `yosys <command>`). `$lib` is the Liberty file from Section 4.2.

Step	Yosys command	Purpose
1	`read_verilog rtl/full_adder.v rtl/mult_array.v`	Read the design
2	`synth -top mult_array -flatten`	Generic synthesis; flatten the full adders into one module
3	`dfflibmap -liberty $lib`	Map flip-flops to SKY130 flip-flop cells
4	`abc -liberty $lib`	Map the logic to SKY130 gate cells
5	`opt_clean -purge`	Remove unused wires
6	`tee -o synth/mult_array_stat.rpt stat -liberty $lib`	Write the cell and area report
7	`write_verilog -noattr synth/mult_array_netlist.v`	Write the gate-level netlist

Run it with `yosys -c synth/synth.tcl`.

Tasks

1. Write and run `synth.tcl`.
2. Fill in the table below from `mult_array_stat.rpt`. The flip-flop cell name contains `dfrtp`; flip-flop area = number of flip-flops × area of one flip-flop cell.
3. Simulate the synthesized netlist with your own testbench to prove synthesis did not change the behaviour:

```
CELLS=$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog
iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls \
  tb/tb_mult.v $CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v synth/
    mult_array_netlist.v
vvp gls
```

Part B results table (fill in)

Metric	Value
Total number of cells	
Number of flip-flops	
Total cell area (μm^2)	
Flip-flop area (μm^2)	
Flip-flop area (% of total)	
The 3 most-used cell types and their counts	
Netlist simulation result (PASS/FAIL line)	

Part B deliverables

- `synth/synth.tcl`, `synth/mult_array_netlist.v`, `synth/mult_array_stat.rpt`.
- Completed table and the netlist simulation PASS line.

8. Part C — Static Timing Analysis with OpenSTA

OpenSTA checks every path between flip-flops against the clock period without simulation. Reference: `doc/OpenSTA.pdf` in the OpenSTA repository. Use the typical corner (`tt_025C_1v80`) only.

Steps your `sta.tcl` must perform

Step	OpenSTA command	Purpose
1	`read_liberty $lib`	Read the cell timing library
2	`read_verilog synth/mult_array_netlist.v` then `link_design mult_array`	Read and link the netlist
3	`create_clock -name clk -period T [get_ports clk]`	Define the clock with period T in ns
4	`set_input_delay 1.0 -clock clk [delete_from_list [all_inputs] [get_ports clk]]`	Input arrival time
5	`set_output_delay 1.0 -clock clk [all_outputs]`	Output required time
6	`report_worst_slack -max`	Print the worst setup slack
7	`report_checks -path_delay max -digits 3 > sta/mult_array_timing.rpt`	Save the critical path report

Run it with `sta sta/sta.tcl`. In a Tcl script, `$lib` must first be set to the Liberty file path, for example `set lib $::env(PDK_ROOT)/sky130A/...`.

How to find the maximum frequency

Run with $T = 10\,\text{ns}$ and note the worst slack WNS (in ns). The minimum period is $\boldsymbol{T_{\text{min}} = T - \text{WNS}}$, and $\boldsymbol{F_{\text{max}} = 1000/T_{\text{min}}}\,\text{MHz}$. Then run again with $T = T_{\text{min}}$ (rounded to 0.01 ns) and confirm the slack is close to 0.

Tasks

1. Find T_{min} and F_{max}.
2. From `mult_array_timing.rpt`, write down the start point and end point of the critical path, and list the cells on it in order.
3. Count how many of those cells come from full adders, and compare with the estimate of about $2N = 8$ full-adder delays from Section 2.

Part C results table (fill in)

Metric	Value
Worst slack at $T = 10\,\text{ns}$ (ns)	
$T_{\min}$ (ns)	
$F_{\max}$ (MHz)	
Slack at $T = T_{\min}$ (ns)	
Critical path start point	
Critical path end point	
Number of cells on the critical path	
Latency (ns) $= 2 \times T_{\min}$	
Throughput (million results per second)	

Part C deliverables

- `sta/sta.tcl` and `sta/mult_array_timing.rpt`.
- Completed table and the list of cells on the critical path.

9. Part D — Physical Design: RTL to GDSII

LibreLane takes your RTL all the way to a layout (GDSII). It runs Yosys, OpenROAD, Magic, KLayout and Netgen for you.

Flow stages

Step	Stage	What it does
1	Synthesis	RTL to standard cells (as in Part B)
2	Floorplan	Sets the chip size and adds the power grid
3	Placement	Places every cell in rows
4	Clock tree synthesis (CTS)	Adds buffers so the clock reaches all flip-flops at nearly the same time
5	Routing	Draws the metal wires between cells
6	Extraction and signoff	Measures wire resistance and capacitance, re-runs timing, checks DRC and LVS
7	GDSII	Writes the final layout file

Settings your `config.json` must contain

Key	Value
`DESIGN_NAME`	`"mult_array"`
`VERILOG_FILES`	`["dir::../rtl/full_adder.v", "dir::../rtl/mult_array.v"]`
`CLOCK_PORT`	`"clk"`
`CLOCK_PERIOD`	$1.2 \times T_{\min}$ from Part C, rounded up to the next 0.5 ns
`FP_SIZING`	`"absolute"`
`DIE_AREA`	`[0, 0, 150, 150]` (a fixed 150 μm $\times$ 150 μm die, needed because the design is very small)

Running the flow

```
cd pnr
librelane config.json                # or:  python3 -m librelane config.json
```

A run takes about 5–15 minutes. Each run creates a folder `runs/<date-time>/`. Its `final/` subfolder holds the GDS file and a metrics file. Copy `final/metrics.json` and the GDS file into `pnr/` for submission. To view the layout, open the GDS file in KLayout (`klayout <file>.gds`). Check the LibreLane documentation if a command or file name differs in your version.

Where to find the numbers

Search `metrics.json` for these words:

Metric	Search for
Number of cells	`instance__count`
Core or die area	`core__area` or `die__area`
Wirelength	`wirelength`
Worst setup slack	`setup__ws`
DRC errors	`drc`
LVS errors	`lvs`

Tasks

1. Write `config.json` and run the flow until it finishes with **0 DRC errors, 0 LVS errors** and setup slack ≥ 0. If timing fails, increase `CLOCK_PERIOD` by 0.5 ns and run again; record every attempt in your README.
2. Fill in the table below.
3. Calculate post-route $F_{\max} = 1000/(\text{CLOCK_PERIOD} - \text{setup slack})$, and compare it with $F_{\max}$ from Part C.
4. Compare the number of cells after place and route with the number after synthesis (Part B). Find out from the reports which kinds of cells were added (look for names containing `buf`, `fill`, `tap` and `decap`).
5. Take two screenshots of the final layout in KLayout: the whole die, and a zoomed view showing standard cell rows and metal wires.

Part D results table (fill in)

Metric	Value
CLOCK_PERIOD used (ns)	
Number of runs needed	
Core area (μm^2)	
Cells after synthesis (from Part B)	
Cells after place and route	
Kinds of cells added	
Wirelength (μm)	
Setup slack (ns)	
Post-route $F_{\max}$ (MHz)	
$F_{\max}$ from Part C (MHz)	
DRC errors	
LVS errors	

Part D deliverables

- `pnr/config.json`, `pnr/metrics.json`, `pnr/mult_array.gds` (do **not** commit the `runs/` folder; it is very large).
- Completed table and the two layout screenshots.

10. Questions for the Report

Answer every question in one short paragraph, using numbers from your own results.

1. The array needs 16 AND gates and 12 full adders. How many cells did synthesis produce for the logic (total cells minus flip-flops)? Explain why the number of cells is not simply $16 + 12$.
2. What is the critical path of your design, from which register to which register? How many cells are on it, and how does this compare with the estimate of about $2N$ full-adder delays?
3. What percentage of the total cell area is flip-flops? Why is this percentage large for such a small design?
4. Why is the post-route $F_{\max}$ lower than the $F_{\max}$ from Part C? Which kinds of cells did the flow add, and why is each kind needed?
5. What does a DRC check verify, and what does an LVS check verify? Why must both pass before a chip can be manufactured?
6. Using the formulas in Section 2, estimate how the number of full adders and the critical path delay would change if the multiplier were 8 bits wide instead of 4. Show your working.

11. Deliverables

☐	Part	Deliverable
☐	All	Repository with exactly the file architecture in Section 5; README with both team members and every command in order
☐	A	`full_adder.v`, `mult_array.v`, `tb_mult.v`
☐	A	PASS line, FAIL line from the error test, waveform screenshot with latency marked, hand-worked example, block diagram
☐	B	`synth.tcl`, netlist, stat report; Part B table; netlist simulation PASS line
☐	C	`sta.tcl`, timing report; Part C table; list of cells on the critical path
☐	D	`config.json`, `metrics.json`, GDS file; Part D table; two layout screenshots
☐	Report	Answers to questions 1–6

12. Grading

Part	Marks
A: RTL design and verification	30
B: Logic synthesis	20
C: Static timing analysis	20
D: Physical design	30
Total	**100**

13. Academic Integrity

- Ensure that you understand the code you submit and are able to explain your implementation and approach.

- External resources and tools may be used to support your learning and development, but you are responsible for understanding, reviewing, and validating the code you submit.
- Your Git history should reflect consistent progress throughout the two-week period. A minimum of 4–5 meaningful commits is expected, showing your progress and development over time.

14. Troubleshooting

Problem	Likely cause	What to do
Docker will not start on Windows	Virtualization or WSL 2 not enabled	Enable virtualization in BIOS and install WSL 2
Testbench prints FAIL	Wrong connection in one row	Check every full adder against the table in Section 2; look at the first mismatch in the waveform
Testbench compares too early	Latency not respected	Wait two rising clock edges after applying inputs before comparing
"Unknown module full_adder"	File missing from the command	Include `rtl/full_adder.v` in every `iverilog` and Yosys command
`.lib` file not found	Wrong PDK selected	Run `echo $PDK_ROOT $PDK`; if needed run `sak-pdk sky130A`
Netlist simulation fails, RTL passes	Registers not reset, or cell model files missing	Reset all registers in `mult_array`; use the exact command in Part B
LibreLane stops with setup violations	Clock period too short	Increase `CLOCK_PERIOD` by 0.5 ns and run again
LibreLane stops at floorplan or power grid	Die area setting wrong	Check `FP_SIZING` is `"absolute"` and `DIE_AREA` is `[0, 0, 150, 150]`
Runs are very slow or crash	Docker has too little memory	Give Docker at least 8 GB RAM and 4 CPUs

If you are stuck on a setup problem for more than two hours, ask your instructor.

15. References

Textbooks

- N. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed., Pearson. Chapter 11 (datapath subsystems: adders and multipliers).
- J. Rabaey, A. Chandrakasan and B. Nikolić, *Digital Integrated Circuits: A Design Perspective*, 2nd ed., Pearson. Arithmetic building blocks and timing.

Tool documentation

Tool	Documentation
Icarus Verilog	steveicarus.github.io/iverilog
GTKWave	gtkwave.github.io/gtkwave
Yosys	yosyshq.readthedocs.io/projects/yosys
OpenSTA	github.com/parallaxsw/OpenSTA (manual: `doc/OpenSTA.pdf`)
LibreLane	github.com/librelane/librelane
OpenROAD	openroad.readthedocs.io
KLayout	klayout.de/doc.html
Magic / Netgen	opencircuitdesign.com/magic, opencircuitdesign.com/netgen
SKY130 PDK	skywater-pdk.readthedocs.io
IIC-OSIC-TOOLS	github.com/iic-jku/IIC-OSIC-TOOLS